



Contents lists available at ScienceDirect

Digital Communications and Networks

journal homepage: www.keaipublishing.com/dcan

A blockchain based privacy-preserving federated learning scheme for Internet of Vehicles

Naiyu Wang^a, Wenti Yang^a, Xiaodong Wang^a, Longfei Wu^b, Zhitao Guan^{a,*}, Xiaojiang Du^c, Mohsen Guizani^d^a School of Control and Computer Engineering, North China Electric Power University, China^b Department of Mathematics & Computer Science, Fayetteville State University, Fayetteville, NC, USA^c Department of Electrical and Computer Engineering, Stevens Institute of Technology, Hoboken, NJ, 07030, USA^d Department of Computer Science and Engineering, Qatar University, Doha, Qatar

ARTICLE INFO

Keywords:

Federated learning
Blockchain
Privacy-preservation
Homomorphic encryption
Internet of vehicles

ABSTRACT

The application of artificial intelligence technology in Internet of Vehicles (IoV) has attracted great research interests with the goal of enabling smart transportation and traffic management. Meanwhile, concerns have been raised over the security and privacy of the tons of traffic and vehicle data. In this regard, Federated Learning (FL) with privacy protection features is considered a highly promising solution. However, in the FL process, the server side may take advantage of its dominant role in model aggregation to steal sensitive information of users, while the client side may also upload malicious data to compromise the training of the global model. Most existing privacy-preserving FL schemes in IoV fail to deal with threats from both of these two sides at the same time. In this paper, we propose a Blockchain based Privacy-preserving Federated Learning scheme named BPFL, which uses blockchain as the underlying distributed framework of FL. We improve the Multi-Krum technology and combine it with the homomorphic encryption to achieve ciphertext-level model aggregation and model filtering, which can enable the verifiability of the local models while achieving privacy-preservation. Additionally, we develop a reputation-based incentive mechanism to encourage users in IoV to actively participate in the federated learning and to practice honesty. The security analysis and performance evaluations are conducted to show that the proposed scheme can meet the security requirements and improve the performance of the FL model.

1. Introduction

The Internet of Vehicles (IoV) is a large-scale integrated network for wireless communication and information exchange between people, vehicles, and roads [1]. In recent years, driven by machine learning, especially deep learning algorithms, vehicle and road data collected by Road Side Units (RSUs) can be used to build a variety of knowledge models [2,3], such as road obstacle analysis models, vehicle misbehavior detecting models, etc. Under this trend, traffic management, vehicle control and other related fields are gradually becoming intelligent, which greatly improves the performance of the Internet of Vehicles [4–6].

However, the data collection may involve highly sensitive personal data [7], such as the location of the vehicle, which can easily cause the participants' privacy concerns and hinder them from sharing data in IoV [8,9]. In addition, the large amount of data transmission may cause network congestion and redundant transmission [10]. Taking all these

factors into account, it is unrealistic to collect IoV data with a centralized server [11,12], which also indicates that traditional machine learning models may not be suitable for distributed IoV with local characteristics [13].

Federated Learning (FL) [14] is considered a potential method to mitigate such concerns, which allows private data to be kept locally. Also, the mode of local knowledge extraction and sharing in FL can speed up the learning process in the complex IoV environment [10]. The aggregation server builds the global model based on the collected local model updates. However, there are still certain risks in this process, studies have shown that the aggregation server may steal sensitive data from the shared models, at the same time, participants may also launch poisoning attacks on the model [15–17].

Madi et al. [18] took into account the unreliability of the aggregator. They proposed to combine Homomorphic Encryption (HE) and Verifiable Computing (VC) techniques to protect the security of the data submitted

* Corresponding author.

E-mail address: guan@ncepu.edu.cn (Z. Guan).<https://doi.org/10.1016/j.dcan.2022.05.020>

Received 31 July 2021; Received in revised form 19 May 2022; Accepted 22 May 2022

Available online 31 May 2022

2352-8648/© 2022 Chongqing University of Posts and Telecommunications. Publishing Services by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

by the client, and verified the integrity of the model issued by the aggregator. Zhao et al. [19] took advantage of the traceability and decentralization of blockchain [20] to replace the aggregator in the traditional FL system, and realized the tracking of malicious user activities. This can largely regulate the behavior of the Model Aggregation (MA) node. However, these works only considered the possible malicious behavior of the aggregator, without taking into account the possible poisoning attacks of the clients to the threat model.

In [21], a hybrid blockchain architecture combined with asynchronous federated learning is proposed to address the problem of unreliable model parameters in the IoV context, and the quality of the parameters is verified through a two-stage verification. However, the possible malicious behavior of the aggregation server has not been dealt with. DeepChain [22] took both types of malicious behaviors into account, and designed a value-driven incentive mechanism based on blockchain for participants' good behaviors. Meanwhile, the threshold Paillier algorithm has been adopted to ensure the privacy of local data. Reasonable incentives are important for mitigating the privacy concerns of clients and attracting them to join federated learning. There have been many studies on incentive mechanisms [23,24], however, they do not handle malicious model updates. Biscotti [25] also proposed a solution to poison attacks by malicious participants, using the Multi-Krum algorithm [26] to reject updates that run counter to the majority of model updates. However, this centralized verification method still faces the problems of single point of failure and high verification cost.

In this paper, we propose the BPFL scheme for privacy-preserving federated learning in IoV, and apply the improved Multi-Krum technology to realize distributed verification and filtering at the ciphertext level. The main contributions in our paper are summarized as follows:

- 1) The additive homomorphic encryption technology is used to encrypt the model updates to provide further privacy protection. Meanwhile, the improved Multi-Krum technology can be closely integrated with the blockchain network, to realize both the filtering of malicious updates and model aggregation at the ciphertext level. In addition, the distributed verification in parallel is enabled to improve the system efficiency.
- 2) We use the Consortium Blockchain as the distributed architecture of FL, and employ the channel technology and the smart contract technology to better achieve privacy protection. All learning processes can be recorded using the immutable ledger.
- 3) We further propose a reputation-based incentive mechanism maintained by Blockchain to encourage more participants with high-quality data to join the federated learning and behave honestly. It can also be used along with the filtering mechanism to assess the contribution of individual participants.

The rest of this paper is organized as follows. In Section 2, the preliminaries are given in. In Section 3, the system model and security model are described. In Section 4, we present the details of the proposed scheme. In Section 5 and 6, the security analysis and performance evaluation are presented, respectively. Section 7 concludes the paper.

2. Preliminaries

2.1. Multi-krum

Multi-Krum is a variant of the Byzantine resilient aggregation rule Krum under multi-worker conditions [26], whose goal is to filter out the vectors that are too far away.

In the Krum aggregation rule $KR(V_1, V_1, \dots, V_n)$, suppose the system can tolerate f Byzantine workers. For any worker i , we define the score as $s(i) = \sum_{i \rightarrow j} \|V_i - V_j\|^2$, where V_j denotes the $n - f - 2$ closest vectors (neighbors) of V_i , and the score is the sum of the Euclidean distances of V_i to V_j .

Following that, Multi-Krum selects the $m \in \{1, 2, \dots, n\}$ vectors $V_1^*, \dots, V_m^*$ which score the best.

2.2. The consistent hashing algorithm

The consistent hashing algorithm [27] is a widely used distributed cache algorithm for scalable system architectures, which organizes the entire hash space (head to end) into a virtual hash ring in a clockwise manner. This algorithm hashes the unique identification symbol of each server to determine its position on the ring. After hashing the key to the data and rotating clockwise from its corresponding position on the ring, the first server encountered is the server to which the data is assigned.

2.3. The paillier algorithm

The Paillier algorithm is one of the most classical additive homomorphic encryption algorithms, which is based on the intractability of the Composite Residuosity Class Problem [28].

- Paillier. $\text{Genkey}() \rightarrow \{(n_0, g), (\lambda, \mu)\}$, where (n_0, g) is the public key and (λ, μ) is the private key. To generate a pair of keys, pick two random large prime numbers p, q , and set $n_0 = p \times q$, $\lambda = \text{lcm}(p-1, q-1)$, then select a base $g \in B$ such that it satisfies $\text{gcd}(L(g^2 \bmod n_0^2), n_0) = 1$ under the definition of $L(x) = (x-1)/n_0$.
- Paillier. $\text{Enc}(m) \rightarrow c = g^m \cdot r^{n_0} \bmod n_0^2$, where the plaintext $m \in \mathbb{Z}_n$ and the random number $r < n_0$.
- Paillier. $\text{Dec}(c) \rightarrow m = \frac{L(c^{\lambda} \bmod n_0^2)}{L(g^{\lambda} \bmod n_0^2)} \bmod n_0$, where the ciphertext $c < n_0^2$.

The paillier algorithm has the property of additive homomorphism: $E(m_0 + m_1) = E(m_0) \cdot E(m_1)$ (E stands for Paillier. Enc).

3. System model and security model

3.1. System model

As shown in Fig. 1, the system model we build for IoV running federated learning consists of the following roles:

Client user: The initializer of a federated learning model proposal. After the system is trained, the final model will be delivered to the client user.

Federated Learning node (FL node): The FL nodes join the system via registration and are responsible for downloading the global model in each round and training the local sample set with the model.

Model Aggregation node (MA node): This kind of nodes belong to the default setting of the system and do not accept external registration. They can act as a bridge between the Client users and the FL nodes, and are responsible for collecting and aggregating the local models to generate the global model for the next round.

Virtual Verification node (VV node): The VV nodes are actually a set of public-private key pairs for the homomorphic encryption generated when the Smart Contract (SC) code is executed. Each VV node has a pair of public and private keys. They perform verification and filtering on the collected local model updates.

Certification Authority (CA): The trusted organization in the system is responsible for the generation, distribution and maintenance of keys and certificates. In addition, CA stores the reputation value of each FL node.

Next, we describe the two major parts of the system model: consortium blockchain and federated learning.

3.1.1. The architecture of consortium blockchain

BPFL is built in a semi-decentralized consortium blockchain structure with two kinds of legal nodes. The FL nodes have only the read access and

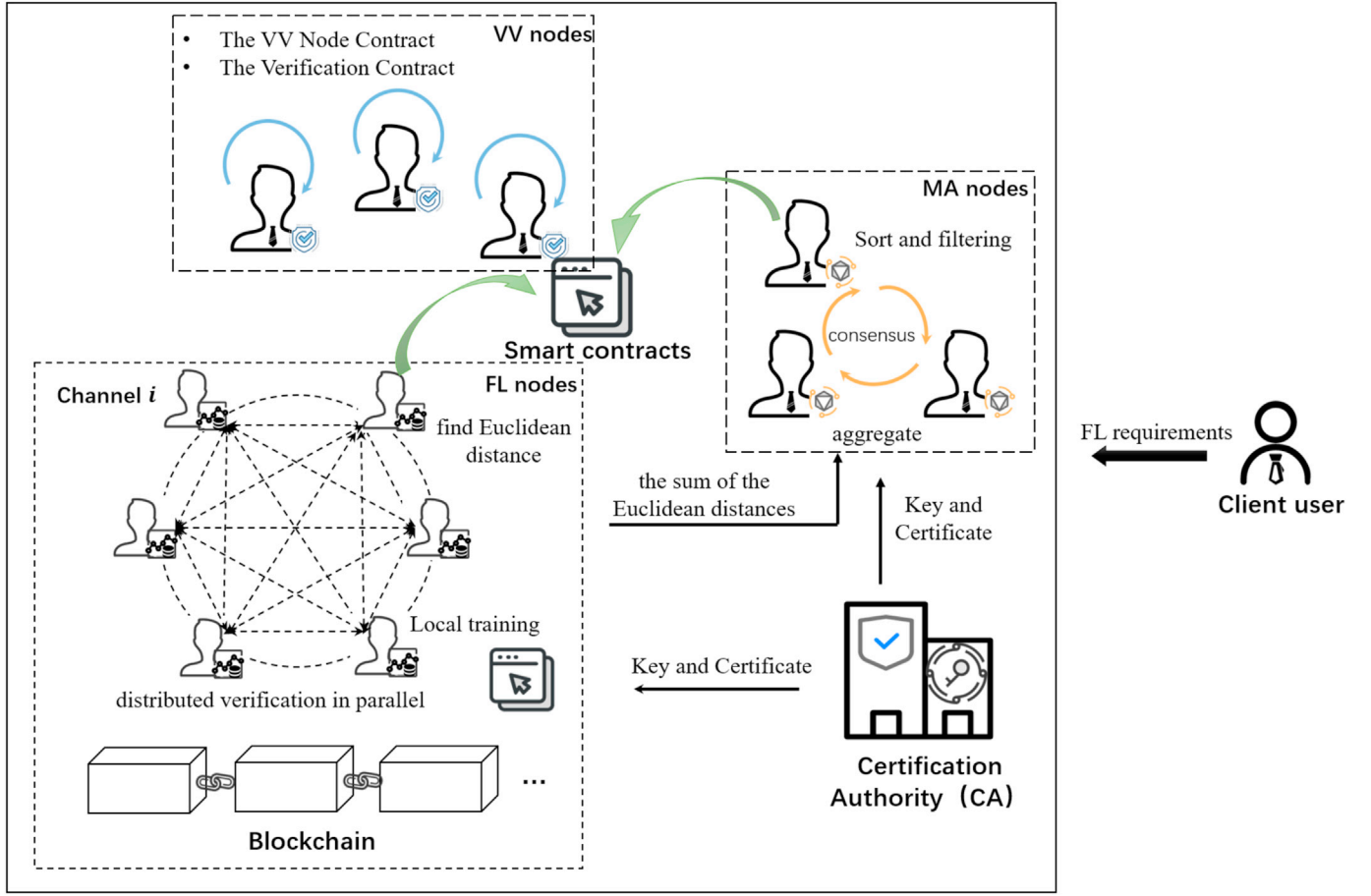


Fig. 1. System model.

need to be authorized by their organizations to join the blockchain with authentication credentials. The MA nodes are the super nodes with the write access, which are responsible for the consensus process.

Each proposal of federated learning initiated by a client user will be the first transaction of the blockchain in a channel, and the block it is located in is the genesis block. The FL nodes can independently choose which channel to join according to the proposal information. Once joined in, they need to participate in the model learning process until the model training is completed. They have no access to the information on the channels that they did not join. The MA nodes are not limited to a specific channel, and are responsible for linking the new block to a specified channel.

The system adopts the Practical Byzantine Fault Tolerance (PBFT) for consensus, assuming that the set of all MA nodes is represented by R , $|R| = 3f + 1$, where f is the maximum number of aggregate Byzantine nodes that can fail validation (To distinguish it from the Byzantine nodes in the multi-krum mechanism, the Byzantine nodes among the MA nodes in the consensus mechanism are called the aggregate Byzantine nodes). The block will be accepted only if more than $2f + 1$ MA nodes reach a consensus successfully.

3.1.2. The architecture of federated learning

BPFL does not restrict the training methods employed by FL. The federated learning is conducted by the cluster of FL nodes $F = \{1, 2, \dots, N_F \in F\}$ within the channel, where $|F| = N_F$. Assume that the local data samples of the i -th FL node F_i is D_i , $|D_i| = N_i$. The generated local model update information $\Delta_i t$ is then sent to the MA node. The global model GM_t of the t -th round is generated by aggregating the local model parameters through the federated averaging algorithm [29].

$$GM_t \leftarrow \sum_{i \in F} \beta_i \Delta_i t \quad (1)$$

where $\beta_i = N_i / \sum_{i \in F} N_i$.

3.2. Threats and goals

We consider the security threats to BPFL from two aspects.

Firstly, we consider the MA node to be honest-but-curious. They honestly perform the aggregation task, but they may try to infer sensitive information from the local model update information.

The second is that the local models might be unreliable. The FL nodes should not be fully trusted even if they have been verified by the CA when joining the system. The reasons are threefold: 1) the local models submitted by FL nodes may come with incomplete training due to laziness; 2) the quality of the data collected may be poor; 3) the node may be impersonated by a malicious node. These challenges cannot be easily solved since the local models submitted by the FL nodes are often encrypted for privacy protection and it is hard to determine whether the local models are inaccurate or fake.

Our BPFL is designed to incorporate model verification to ensure the effectiveness on the basis of privacy protection.

4. Details of the proposed scheme

The main notations in this section are summarized in Table 1.

4.1. Algorithm detail

4.1.1. System initialization

When the system is initialized, it contains MA nodes, each of which is

Table 1
Summary of main notations.

Acronym	Description
N_v	the current number of VV nodes
N_i	the number of F_i 's local samples
N_F	the number of FL nodes in a channel
L_{ID}	The serial number of the FL proposal
T_w	The upper bound on the waiting time for model confirmation
GM_t	global model for the t -th iteration
LM_t^i	local model of F_i for the t -th iteration
Mu_t^i	the model update of F_i for the t -th iteration
u_t^i	the processed model update of F_i for the t -th iteration
f	number of aggregate Byzantine nodes
λ	the ratio of local model parameters to be retained
ϵ	Filtering ratio of Byzantine nodes
r_h, r_m	the honesty and malicious thresholds
r_t	the reputation threshold of a FL process

configured with the corresponding public and private keys as well as the certificates. The keys and certificate held by the i -th MA node are expressed as $(Pk_{M_i}, Sk_{M_i}, Cert_{M_i})$. When a FL node with legitimate credentials joins the system, it needs to submit an application for registration to CA. After successful authentication, it can receive its public key, private key, and certificate $(Pk_{F_i}, Sk_{F_i}, Cert_{F_i})$.

The Initial Function of VV node contract is executed as shown in Algorithm 1, where N_v represents the current number of VV nodes. The contract will generate a batch of public and private keys for the homomorphic encryption. And the consistent hashing algorithm will be used to locate each VV node on the hash ring based on its public key, as shown in Fig. 2, where v_i represents the i -th VV node. This function will be performed periodically during the operation of the system to reset the current VV nodes. All the key pairs of VV nodes are submitted to the CA for storage along with the keys of other types of nodes and are updated upon reset.

Algorithm 1: The VV Node Contract

Initial Function:

Input: N_v
Output: The hash ring of the VV nodes
 (1)for $i = 0$ to N_v do
 $v_i.pubkey, v_i.seckey =$
 $Paillier.Genkey();$
 $V \leftarrow v_i;$
 (2)for $i = 0$ to N_v do
 $v_i.addr \leftarrow Hash(v_i.pubkey);$
 $Sort(V) \leftarrow Addr_v =$
 $AscendSort(v_i.addr);$

Select Function:

Input: $hash$
Output: sv
 (1) $addr \leftarrow hash$
 (2)if $addr > Addr_v[N_v - 1]$ then
 $sv = v_0;$
 (3)else if $Addr_v[i - 1] < addr < Addr_v[i]$
 then
 $sv = v_i;$
return $sv;$

4.1.2. Proposal initiation and response

The client user submits a federated learning proposal to the system, including the required learning size threshold ls_b , model convergence conditions con (can be a restriction on the number of model iterations or on the accuracy of the global model), and the initial global model GM_0 . The proposal will be submitted to a nearby MA node M_k .

M_k first needs to make sure that enough honest FL nodes are involved in the proposal. After adding a serial number L_{ID} to the proposal, it signs the proposal and broadcasts $sig_{Sk_{M_k}}[L_{ID}||con]$ to the system. Upon receiving the broadcast, the FL nodes in the system first validate the signature. Then, the FL nodes decide whether to participate in the training based on the local data set as well as the computation and communication costs. If so, it responds to the transaction by sending the number of local samples N_i and the signature of the serial number $sig_{Sk_{F_i}}(L_{ID})$ to M_k .

4.1.3. Proposal confirmation and local model update

Proposal confirmation: The system sets a response wait time T_w . The MA node M_k collects the responses within T_w after the broadcast, validates the identity of each node that has responded, and classifies them according to L_{ID} . As long as the number of validated nodes is greater than ls_b , the proposal can be confirmed, otherwise, the notice of response failure will be returned to the client user.

Create channel: Once the proposal is confirmed, M_k needs to record N_i of F_i under the corresponding L_{ID} . Then it hashes the confirmed proposal as the input to the Select function of the VV node contract to select the VV node of the first-round. As shown in Algorithm 1, the public key Pk_{v_i} of the selected node v_i , L_{ID} and the proposal information, will be signed by M_k , $Sig_{Sk_{M_k}}(L_{ID}||con||ls_b||GM_0||Pk_{v_i})$. Then the PBFT consensus process is executed by the MA nodes. Once more than $2f + 1$ MA nodes confirm the transaction (f is the number of aggregate Byzantine nodes in the blockchain system), the block can be used as the genesis block in the new channel to start a new blockchain. The set of responded and validated FL nodes can be added to the corresponding channel, which are represented as $F = \{1, 2, \dots, N_F \in F\}$, where $|F| = N_F$.

Local model update: Before the first iteration, the FL node in the channel reads GM_0 as well as v_i in the genesis block to start training. In round t , the FL node F_i which is selected to participate in this round of training downloads the previous round of global model GM_{t-1} and Pk_{v_i} . F_i conducts model training according to the local data samples and obtains the local model LM_t^i . The difference between the local model LM_t^i and global model GM_{t-1} is the model update by F_i in round t : $Mu_t^i = LM_t^i - GM_{t-1}$. In order to reduce the communication overhead of the interactions between nodes, we set the ratio of parameters to be retained to λ , the part of parameters $\lambda \times |Mu_t^i|$ with the largest absolute value is selected to keep the original value, while the rest of Mu_t^i , $(1 - \lambda) \times |Mu_t^i|$, is set to zero. According to the proof in Ref. [30], retaining partial model update will not affect the accuracy of the final model.

The processed model update of F_i is expressed as u_t^i . To facilitate subsequent filtering, u_t^i and $-u_t^i$ are encrypted with the public key respectively to get $E_{v_i}(u_t^i)$, $E_{v_i}(-u_t^i)$. More specifically, in order to calculate the gap between the model updates submitted by each FL node, we need to find the Euclidean distance between the ciphertexts. On this basis, we propose a distributed verification strategy. By encrypting $-u_t^i$, we prepare for the subsequent subtraction in advance, so that each node can perform the verification preparation work synchronously. Then F_i broadcasts $Sig_{Sk_{F_i}}[E_{v_i}(-u_t^i)]$ to the other FL nodes in the channel.

4.1.4. Model verification and filtering

The ciphertext-level model filtering mechanism is integrated with the homomorphic encryption algorithm, which can work well with the P2P network in blockchain. This improves Multi-Krum [21,25,26] by changing its centralized verification to distributed verification, enabling to achieve a certain degree of parallel execution. After F_i receives the broadcast message from other responding FL nodes in round t , the

identity validation of those FL nodes is carried out. The set of $E_{v_i}(-u_i^t)$, $E_{v_i}(u_i^t)$ and Pk_{v_i} are then used as inputs to invoke the Sum function of the Verification Contract, which is shown in **Algorithm 2**.

Firstly, the distance between each two model updates is calculated (**Algorithm 2, SUM-(1)**), which will serve as an important basis for subsequent filtering. Since the FL nodes calculate the model update ciphertext locally in advance, they can easily perform verification locally and synchronously. Compared with the centralized verification model, the computing tasks are distributed to each FL node, which can not only avoid huge computing pressure on one node, but also improve the verification efficiency. In addition, from a security point of view, since each node receives a set of ciphertexts and does not have the private key to decrypt, it has no idea about how its own results compare with other nodes in the set.

Then, the contract sends Pk_{v_i} to the CA and obtains the corresponding private key (**Algorithm 2, SUM-(2)**). Then *Paillier.Dec* is executed to decrypt all distances, and calculates the corresponding Euclidean distances (**Algorithm 2, SUM-(3)**). At this point, according to the additive homomorphism property, we have

$$\text{Paillier.Dec}(E[Dis_i[j]]) = u_i^t - u_j^t \quad (2)$$

$$Dis_i[j] = \|u_i^t - u_j^t\|^2 \quad (3)$$

where $Dis_i[j]$ represents the distance between the model updates of F_i and F_j .

In the next step of execution (**Algorithm 2, SUM-(4)**), the contract will calculate Dis_{sum-i} , the sum of the Euclidean distances between F_i and all other FL nodes. The output is then sent to M_k together with $E_{v_i}(u_i^t)$ after F_i signs.

Algorithm 2: The Verification Contract

Sum Function:

Input: $E_{v_i}(u_i^t)$, a set of $E_{v_i}(-u_i^t)$, Pk_{v_i}
Output: Dis_{sum-i}
 (1) **for** $j = 0$ to N_v , $j \neq i$ **do**
 $Edis_i[j] = E_{v_i}(u_i^t) \cdot E_{v_i}(-u_j^t) = E_{v_i}(u_i^t - u_j^t)$;
 (2) $Pk_{v_i} \rightarrow CA \rightarrow Sk_{v_i}$
 (3) **for** $j = 0$ to N_v , $j \neq i$ **do**
 $Ddis_i[j] = \text{Paillier.Dec}(Edis_i[j])$;
 $Dis_i[j] = \|Ddis_i[j]\|^2$;
 (4) $Dis_{sum-i} = \sum_{j \in [0, n], j \neq i} Dis_i[j]$

Filter Function:

Input: a set of Dis_{sum-i}
Output: $ValidU$
 (1) $SDis = \text{AScendSort}(Dis_{sum-i})$
 (2) $ValidDis = SDis.subset[0, n - \epsilon n]$
 (3) **for** $i = 0$ to N_v **do**
 if $Dis_{sum-i} \in ValidDis$ **then**
 $ValidU.add(i)$;
return $ValidU$

The task of M_k at this time is to call the Filter Function of the Verification Contract with each FL node's Sum Function output Dis_{sum-i} . Specifically, it first verifies the identity of each FL node. Then, it ranks the

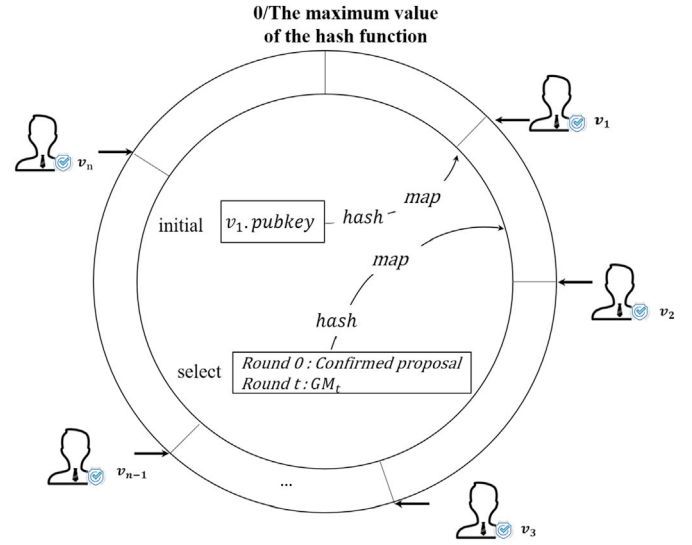


Fig. 2. Initialization and selection of VV nodes.

set of Dis_{sum-i} in the ascending order (**Algorithm 2, Filter-(1)**). Suppose the filtering ratio of Byzantine FL nodes in the channel is ϵ , the first $n - \epsilon n$ values in the ranked set are selected (**Algorithm 2, Filter-(2)**). That is, we will select and keep the $n - \epsilon n$ nodes whose updates are the closest to other models updates, and the rest will be rejected. Here, we consider the unified collection of M_k in the consortium blockchain, and ignore the distinction between neighbor and non-neighbor nodes in the original Multi-Krum. Then, we add the serial numbers of the reserved model updates into the collection $ValidU$ as the output (**Algorithm 2, Filter-(3)**).

To perform the final aggregation of the global model in this round, M_k reads the number of samples N_i for each reserved FL node F_i . The set of N_i will be the input of the Aggregate Contract, along with Pk_{v_i} , $ValidU$ and the set of $E_{v_i}(u_i^t)$, as shown in **Algorithm 3**. Firstly, the total number of samples retained in the current round is calculated. Then, the Federated Averaging algorithm [29] is executed using the serial numbers in the $ValidU$ set. Finally, the *paillier.Dec* algorithm performs the final decryption operation on the ciphertext of the global model to obtain the global model GM_t of the current round t .

Algorithm 3: The Aggregate Contract

Input: Pk_{v_i} , $ValidU$, a set of $E_{v_i}(u_i^t)$ and a set of N_i
Output: $GM_t, E_{\Delta GM}$
 (1) $Pk_{v_i} \rightarrow CA \rightarrow Sk_{v_i}$
 (2) $N^t = \sum_{k \in ValidU} N_k$
 (3) $E_{\Delta GM} = \sum_{k \in ValidU} \frac{N_k}{N^t} E_{v_i}(u_i^t)$
 (4) $GM_t = GM_{t-1} + \text{Paillier.Dec}(E_{\Delta GM})$
return $GM_t, E_{\Delta GM}$

4.1.5. Consensus and Global Model Update

Prior to saving the global model GM_t to the block, the MA node M_k needs to test the convergence conditions *con* of the global model. If this condition is not met, M_k hashes the GM_t and the local model update ciphertext of the reserved FL nodes. The Select function in **Algorithm 1** is then executed to choose the VV node of the next round, and the input of the Select function is the hash result of GM_t . The output VV node's public key Pk_{v_i} , GM_t and the timestamp are added to the block, along with the result of the hash operation, as shown in **Fig. 3**. Upon completion of the block construction, the consensus process can start. The new block will be linked to the tail of the blockchain after the validation.

This process of derivation from the *Local Model Update* to the *Global Model Update* will be repeated until the convergence conditions proposed by the client user are met.

4.1.6. Return the final model

Once the model has converged, M_k first hashes the model ciphertext of the last round. Then the hash results together with the ciphertext of the global model $E_{\Delta GM}$ and the timestamp are added to the block, as shown in the dotted area of Fig. 3. After the consensus is passed, the new block is added to the blockchain. In the final stage of the federated learning, M_k returns the global model GM_t to the user with its own signature as the final result of the proposal.

4.2. Incentive mechanism based on reputation

To motivate the owners of IoV data to actively participate in the federated learning and to provide high quality data, a reputation-based incentive mechanism is designed in BPFL based on the results of the above filtering mechanism. The description of the incentive mechanism is presented below.

4.2.1. Reputation initialization

The system initializes a reputation value for each FL node that joins, which is stored in the CA along with the secret keys and certificates. Assume that the reputation values are integers in a finite set $[0, r_{\max}]$, and r_h, r_m are the honest and malicious thresholds within this range respectively. If the reputation value r of a node has $r \geq r_h$, we consider the node to be honest. Similarly, if $r < r_m$, we consider the node to be malicious. For malicious nodes, their certificates will be revoked. We initialize the reputation value of the new node to the honesty threshold.

4.2.2. Reward publishing and thresholding

The client user, as the publisher of the learning task, needs to estimate the benefits of the outcome model and attach rewards rew to the proposal. This not only affects the willingness of FL nodes to participate in the federated learning, but also indicates the level of contribution made by the FL nodes. Moreover, the user also needs to determine the reputation threshold r_t based on the required learning size threshold l_s . This threshold is used to judge whether the FL node is eligible to participate in the training according to its reputation value, and a node F_i can join the training only if $r_i \geq r_t$ is satisfied. When the sensitivity of the model to be trained is high, the client user must set a high threshold; but in the circumstance that the model needs a large sample size and has low sensitivity, a smaller threshold can be set. The MA node will then add this information to the training proposal.

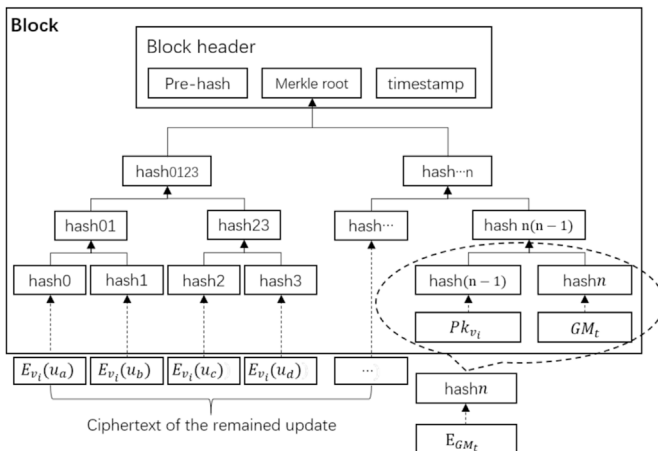


Fig. 3. The structure of the update block.

4.2.3. Reputation validation

When the MA node validates the response of the FL node F_i , it needs to ask CA for its reputation value and compare it with r_t . Only if the number of nodes whose reputation values exceeds the required learning size threshold l_s , can the proposal be finally confirmed.

4.2.4. Reward allocation and reputation renewal

During the training process, we initialize the contribution value cb_i of each FL node F_i to 0, which is maintained by the MA node M_k . In the t -th round, after M_k receives the output of **Algorithm 2-Filter ValidU**, the contribution value of each node is updated as follows:

$$\begin{cases} cb_i = cb_i + 1, i \in \text{ValidU} \\ cb_i = cb_i, i \notin \text{ValidU} \end{cases} \quad (4)$$

At the end of model training, assuming there are t_s rounds, the rewards of different nodes are as follows:

$$\begin{cases} r_i = r_i - 1, \text{rew}_i = 0, cb_i < 2t_s/3 \\ r_i = r_i + 1, \text{rew}_i = \frac{cb_i}{\sum_{cb_i \geq 2t_s/3} cb_i} \text{rew}, cb_i \geq 2t_s/3 \end{cases} \quad (5)$$

5. Security analysis

We will analyze the security of the system from three aspects. First, we will explain the security guarantee brought by the underlying consortium blockchain. Second, we will analyze the security of the VV nodes' keys maintained by the smart contract. On the basis of key security, the privacy protection effectiveness of BPFL is analyzed.

5.1. Security of information on the blockchain

As a permissioned blockchain, the consortium blockchain can initially screen out malicious adversaries through the access mechanism. Secondly, BPFL restricts each FL process to a channel, and nodes outside the channel cannot access the information inside the blockchain, thus achieving data isolation among different FL processes. Thirdly, the cryptographic algorithms adopted by the blockchain, including the elliptic curve signature and Merkle tree can also ensure the security of data. Finally, the ledger can clearly record the training process of each model, so as to facilitate investigation when problems are detected.

5.2. Security of the VV nodes' key

We will demonstrate the security of the keys throughout their life cycles. In the VV node contract, after initializing the hash ring, none of the private keys of the nodes are returned to the MA node. We also periodically reset the hash ring, which further enhances the security of the private keys. The subsequent decryption process is also encapsulated within the contract, so neither the FL node nor the MA node could get the plaintext of the local model in the whole iteration process. Since the smart contracts cannot be changed after deployed, the risk of keys being tempered with is negligible.

5.3. Privacy protection in the model

First of all, each FL node needs to carry out homomorphic encryption on the local model parameters. For an honest-but-curious MA node, it can only read the ciphertext, and has no idea about the private key to the selected VV node. According to the intractability of Composite Residuosity Class Problem in Paillier algorithm [28], the MA node cannot decrypt the ciphertext in polynomial time. For the malicious participant who knows the public key used for encryption, likewise, he cannot infer the private key from the public key, so he cannot obtain the content of model update from the ciphertext of other nodes. Additionally, based on

Table 2
Dataset basic settings.

Items	Datasets		
	Mnist	BelgiumTS [31]	Emnist [32]
Size of train/test set	60000/ 10000	4575/2520	341,873/ 40,832
Label classes	10	62	10
local training times	1	5	3
Number of clients	10	10	3383 ^a
Number of samples for each client	6000	457	/

Note: ^a There are 3383 clients in the dataset, and we selected 500 of them for experiments.

the unidirectional nature of hash functions, it is clear that a malicious party cannot steal any sensitive information from the hash values.

6. Performance evaluation

In this section, we evaluate the performance of BPFL from three aspects: the model accuracy, the computation overhead of the verification and the effectiveness of the incentive mechanism.

6.1. Experiment setup

Our simulation experiments are conducted on a computer with Intel Core i7 CPU at 3.79 GHz and 16.0 GB RAM running Ubuntu 18.04 LTS

2.5 GB OS. We build the federated learning environment with Python(v-3.6.3), TensorFlow(v-2.4.2) and TensorFlow-federated(v-0.17.0). We build the blockchain on the Hyperledger Fabric (v-1.0) platform. For simplicity, we conduct simulations in one channel. We use Go (v-1.10.3) and its associated libraries to write the smart contracts.

The basic settings of the datasets used in the experiments are shown in Table 2. The first two rows of the table are the basic information of the datasets, and the last three rows are our settings in the experiment. We shard mnist and BelgiumTS to simulate client samples in FL. We first conduct experiments on the classical dataset Mnist, for which we choose a Convolutional Neural Network (CNN) with two convolutional layers and two max-pooling layers using Rectified Linear Unit (ReLU) activations. We then evaluate the proposed model in a more general scenario for the Internet of Vehicles, the samples are from the Belgian Traffic Signs dataset(BelgiumTS) [31]. We use a CNN with five convolutional layers and three max-pooling layers for BelgiumTS. This is trained with categorical_crossentropy loss, optimized with the Adam optimizer.

In order to further reflect the non-Independently identical distribution (Non-IID) characteristics of federated learning, we also evaluate the model with the Emnist dataset [32], which is a federated version of the MNIST dataset. We selected a sample of 500 clients(a total of 47981 samples), each of which has a different sample size and we assign the weight of aggregation according to the sample size. Considering the dynamic nature of the vehicles [1], all clients may not be online at the same time. We randomly select 20% of the clients for training in each round. The model we choose is a CNN with two convolutional layers and two max-pooling layers, optimized with the Adam optimizer.

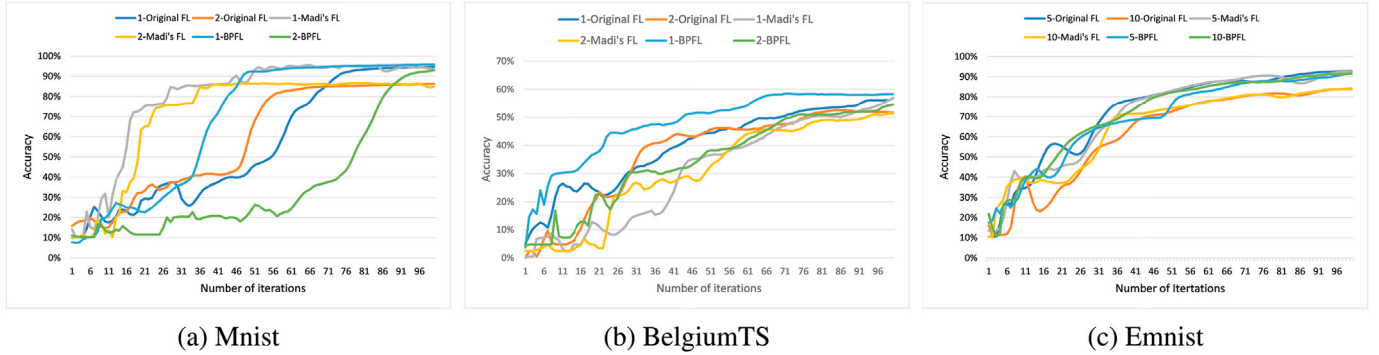


Fig. 4. Comparison of model accuracy. We simulated a federated learning scenario with 10 clients in the Mnist dataset and the BelgiumTS dataset as shown in Tab.2. The “x-” represents that the number of malicious clients in the set is x. The label flip ratio of malicious clients in the experiment on the Emnist dataset is 100%. In the case of one malicious client in the Mnist dataset, the label flip ratios are 50%. While in the two malicious clients situation, the ratios of the two clients are 50% and 70%, respectively. Similarly, the label flip ratios of the above two cases in the BelgiumTS dataset are 70%, 70%/90%. Since $\epsilon = 0.1$ in BPFL, one client is filtered out for Mnist and BelgiumTS, and ten clients are filtered out for Emnist.

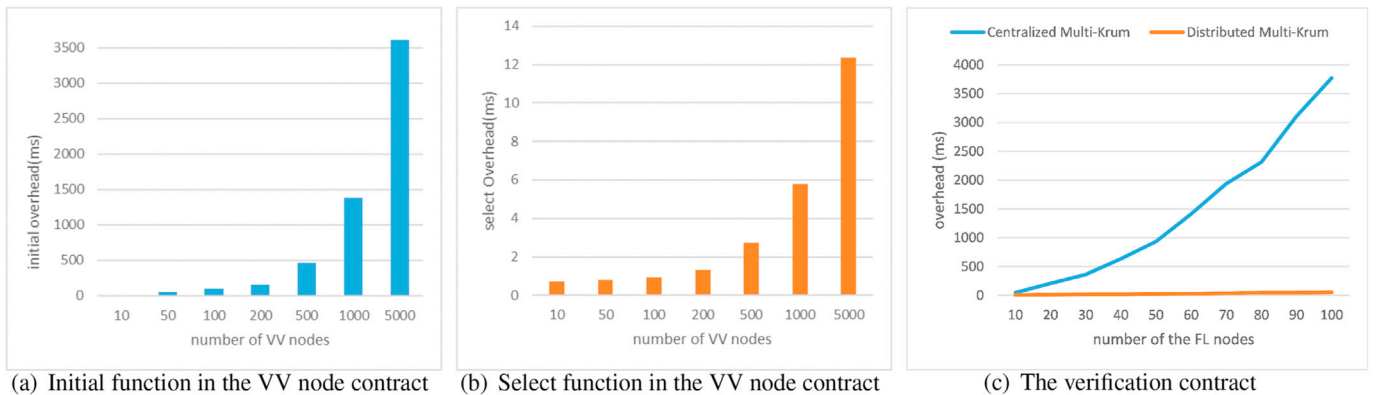
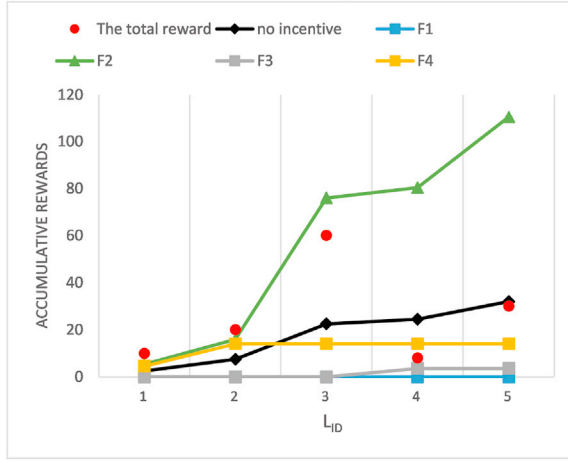
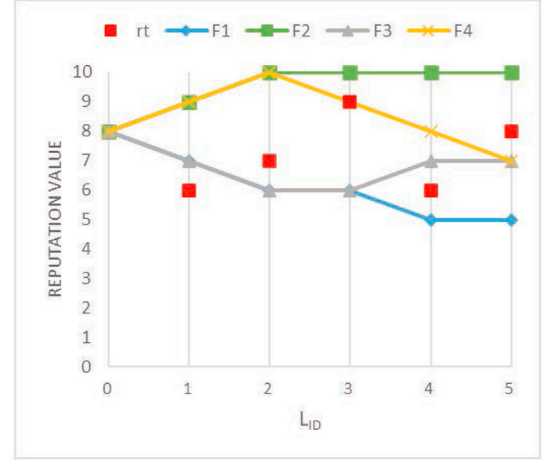


Fig. 5. Computation overhead for the Verification.



(a) Comparison of accumulative rewards



(b) Comparison of reputation

Fig. 6. Evaluation of incentive mechanism.

Table 3

The contribution value of each node.

L_{ID}	F1	F2	F3	F4
1	30	95	60	80
2	50	94	65	85
3	/ ^a	96	/	66
4	45	92	75	50
5	/	98	/	43

^a Note: '/' indicates $r_i < r_b$, failure to participate in training.

6.2. Model accuracy

To test the performance of BPFL model filtering on model accuracy, we first simulate a label flipping attack on the three datasets, that is, flipping the sample labels of some clients to act as malicious clients. We compare the differences in model accuracy after 100 iterations between BPFL, Madi's homomorphic encryption (Paillier cryptosystem) scheme [18] (for the sake of fairness, we discard the Verifiable Computation in the scheme), and the original federated learning [14] under two scenarios for each dataset. Referring to the scheme in Ref. [30], we set $\lambda = 0.2$ for each FL node when selecting the proportion of model update parameters after experimental verification. The experiments set the filtering ratio of Byzantine FL nodes $\epsilon = 0.1$ and assume that the number of participating FL nodes is N_{FL} , which means that we filter out $\lfloor \epsilon \times N_{FL} \rfloor$ updates that are considered malicious. For example, in a scenario with 10 clients, we will filter out one model update.

From the comparison of the three different schemes shown in Fig. 4(a), we can find that the more malicious FL nodes there are, the more negative impact they pose on the accuracy of the model. Referring to the parameter accuracy settings in Ref. [18], we can see that the accuracy of madi's FL model is not much different from that of the original FL, which shows that the loss caused by the combination of federated learning and homomorphic encryption is acceptable. At the same time BPFL malicious screening mechanism can effectively get rid of the negative effects caused by the dishonest nodes, which outperforms that of FL [14,18]. In Fig. 4(b), the number of samples in the dataset is small, but in spite of this, it can still be seen that the screening mechanism of BPFL can effectively improve the accuracy of the model. And it can be seen from the "2-BPFL" of Fig. 4(a)(b) that in the case of a high proportion of malicious clients, BPFL can maintain the model accuracy within a relatively good level by filtering out the client with the most malicious influence.

To test the performance of BPFL model filtering on the non-IID dataset, we conduct similar tests on the Emnist dataset, where the sample labels of the introduced malicious clients are completely flipped due to the small sample size of its clients. We can conclude from Fig. 4(c) that

the fluctuations in the model training process are large and the difference in the accuracy distribution of different models is small, which is mainly due to the uneven sample quality of each client in the dataset and the randomness of the client-side selection algorithm. However, Even in this case, the screening mechanism can still play its role. In addition, by comparing the "5-BPFL" and the "10-BPFL" cases, we found that if the set screening rate is high, some local updates of honest clients will be screened out, which will weaken the advantage of BPFL to some extent. On the other hand, a very low screening rate will let some malicious clients pass, which has a negative impact on model training. Therefore, it is crucial to set an appropriate screening ratio. We generally recommend that client users increase a reputation threshold as much as possible while keep a low screening ratio to reduce the possibility of malicious users participating in training.

6.3. Computation overhead of the verification

We evaluate the computational overhead of two smart contracts related to the verification process.

6.3.1. Overhead of the VV node contract

We use SHA-256 for the hash function in the VV node contract. For the Initial function and the Select function, we respectively test the runtime overhead under different number of VV nodes, as shown in Fig. 5(a)(b). We can see that the time for executing the Select function in the VV node contract is very small compared to the time for the Initial function, and that the overhead of executing the Initial function is acceptable in the case of thousands of key pairs. Since most of the calls on the VV node contract are for the Select function, it can be concluded that the time of selecting the key in each round does not cause additional overhead.

However, when the contract is running in the Fabric system, since the operation time of the chaincode to the chain is much longer, the computational overhead of this part is constantly around 1.3 s regardless of how the number of keys changes.

6.3.2. Overhead of the verification contract

We test the overhead of BPFL for calling the Verification contract using the MNIST dataset, where $\lambda = 0.2$ is set for each FL node when selecting the proportion of model update parameters [30] (Section IV-A-3). We compare the overhead of the original centralized Multi-krum algorithm with our improved algorithm in BPFL. The centralized Multi-krum algorithm utilizes the combination of multi-krum and federated learning in Biscotti [25]. In order to highlight the effect of the distributed verification,

the differential privacy part in Ref. [25] is omitted and we only focus on the multi-krum-based central verification method. To ensure fairness, use the Paillier cryptosystem in the experiments of both algorithms. we can see from Fig. 5(c) that the distributed parallel verification of our improved algorithm in BPFL can greatly reduce the runtime overhead of the verification contract. In addition, the total overhead increases slowly as the number of FL nodes increases from 10 to 100. This is mainly because the most time-consuming calculations of ciphertext distance and the decryption calculations are performed in a distributed and parallel manner.

6.4. Evaluation on the incentive mechanism

For the convenience of observing the effect of the incentive mechanism, we set the maximum reputation value $r_{\max} = 10$, and the honest threshold $r_h = 8$. Assume that four types of FL nodes participate in five FL trainings, $L_{ID} = \{1, 2, \dots, 5\}$. The four types of nodes are F1: dishonest node, F2: honest node, F3: from dishonest to honest, and F4: from honest to dishonest. The reward and reputation threshold for each FL are drawn with the red scatter dots in Fig. 6 (a)(b), respectively. We set 100 rounds for each FL, and the contribution value of each node is shown in Table 3.

As can be seen from Fig. 6, BPFL can better allocate rewards and track the performance of nodes through reputation than the scheme without incentives. For the honest node, it can not only get the most rewards but also have a greater probability to participate in trainings with high reward values. Therefore, our proposed incentive mechanism can effectively motivate FL nodes to join the system and conduct honest behaviors.

7. Conclusion

In this paper, in order to address the security threats faced by the privacy-preserving federated Learning schemes for IoV, we proposed BPFL, which can verify and filter local model updates with privacy protection. BPFL is built with the consortium blockchain as the underlying distributed framework. Employing the homomorphic encryption and the improved Multi-Krum technology, BPFL can encrypt local model updates, and perform model verification and filtering at the ciphertext level. Particularly, the distributed parallel verification in BPFL can greatly reduce the runtime overhead. We also designed a reputation-based incentive mechanism to encourage users with high-quality data to actively participate in FL and to behave honestly. Our future work will focus on communication issues due to vehicle dynamics, which mainly include two aspects: 1) Design an asynchronous federated learning model incorporating the potential straggler problem. 2) Design a flexible client adaptive selection strategy with regard to the above communication overhead and delay issues, to improve the adaptability of the proposed scheme in the IoV.

Declaration of competing interest

The authors declare that there is no conflict of interests regarding the publication of this paper.

Acknowledgements

The work is supported by the National Natural Science Foundation of China under Grant 61972148.

References

- [1] X. Lin, J. Wu, S. Mumtaz, S. Garg, J. Li, M. Guizani, Blockchain-based on-demand computing resource trading in iov-assisted smart city, *IEEE Transactions on Emerging Topics in Computing* 9 (3) (2020) 1373–1385.
- [2] P. Liu, H. He, T. Fu, H. Lu, A. Alalaiwi, M.W.I. Wasi, Task offloading optimization of cruising uav with fixed trajectory, *Comput. Network.* 199 (2021) 108397.
- [3] T. Fu, C. Wang, N. Cheng, Deep-learning-based joint optimization of renewable energy storage and routing in vehicular energy network, *IEEE Internet Things J.* 7 (7) (2020) 6229–6241.
- [4] D.M. Manias, A. Shami, Making a case for federated learning in the internet of vehicles and intelligent transportation systems, *IEEE Network* 35 (3) (2021) 88–94.
- [5] J.S. Ng, W.Y.B. Lim, H.-N. Dai, Z. Xiong, J. Huang, D. Niyato, X.-S. Hua, C. Leung, C. Miao, Communication-efficient federated learning in uav-enabled iov: a joint auction-coalition approach, in: *GLOBECOM 2020-2020 IEEE Global Communications Conference*, IEEE, 2020, pp. 1–6.
- [6] D. Wang, B. Song, D. Chen, X. Du, Intelligent cognitive radio in 5g: ai-based hierarchical cognitive cellular networks, *IEEE Wireless Commun.* 26 (3) (2019) 54–61.
- [7] A. Upreti, D.B. Rawat, J. Li, Privacy preserving misbehavior detection in iov using federated machine learning, in: *2021 IEEE 18th Annual Consumer Communications & Networking Conference (CCNC)*, IEEE, 2021, pp. 1–6.
- [8] M. Shen, Y. Deng, L. Zhu, X. Du, N. Guizani, Privacy-preserving image retrieval for medical iot systems: a blockchain-based approach, *IEEE Network* 33 (5) (2019) 27–33.
- [9] W. Yang, N. Wang, Z. Guan, L. Wu, X. Du, M. Guizani, A Practical Cross-Device Federated Learning Framework over 5g Networks, *arXiv preprint arXiv: 2204.08134*.
- [10] Y. Zou, F. Shen, F. Yan, J. Lin, Y. Qiu, Reputation-based regional federated learning for knowledge trading in blockchain-enhanced iov, in: *2021 IEEE Wireless Communications and Networking Conference (WCNC)*, IEEE, 2021, pp. 1–6.
- [11] S.R. Pokhrel, J. Choi, Improving tcp performance over wifi for internet of vehicles: a federated learning approach, *IEEE Trans. Veh. Technol.* 69 (6) (2020) 6798–6802.
- [12] P. Liu, Y. Ding, T. Fu, Optimal throwboxes assignment for big data multicast in vdnets, *Wireless Network* (2019) 1–11.
- [13] X. Lin, J. Wu, A.K. Bashir, J. Li, W. Yang, J. Piran, Blockchain-based incentive energy-knowledge trading in iot: joint power transfer and AI design, *IEEE Internet Things Journal* (99) (2020) 14685–14698, <https://doi.org/10.1109/JIOT.2020.3024246>, 1–1.
- [14] H. B. McMahan, E. Moore, D. Ramage, B. A. y Arcas, Federated Learning of Deep Networks Using Model Averaging, *arXiv preprint arXiv:1602.05629*.
- [15] N. Wang, W. Yang, Z. Guan, X. Du, M. Guizani, Bpfl: a blockchain based privacy-preserving federated learning scheme, in: *2021 IEEE Global Communications Conference (GLOBECOM)*, IEEE, 2021, pp. 1–6.
- [16] R. Shokri, M. Stronati, C. Song, V. Shmatikov, Membership inference attacks against machine learning models, in: *2017 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2017, pp. 3–18.
- [17] X. Du, Qos routing based on multi-class nodes for mobile ad hoc networks, *Ad Hoc Netw.* 2 (3) (2004) 241–254.
- [18] A. Madi, O. Stan, A. Mayoue, A. Grivet-Sébert, C. Gouy-Pailler, R. Sirdey, A secure federated learning framework using homomorphic encryption and verifiable computing, in: *2021 Reconciling Data Analytics, Automation, Privacy, and Security: A Big Data Challenge (RDAAPS)*, IEEE, 2021, pp. 1–8.
- [19] Y. Zhao, J. Zhao, L. Jiang, R. Tan, D. Niyato, Z. Li, L. Lyu, Y. Liu, Privacy-preserving blockchain-based federated learning for iot devices, *IEEE Internet Things J.* 8 (3) (2020) 1817–1829.
- [20] N. Wang, X. Zhou, X. Lu, Z. Guan, L. Wu, X. Du, M. Guizani, When energy trading meets blockchain in electrical power system: the state of the art, *Appl. Sci.* 9 (8) (2019) 1561.
- [21] Y. Lu, X. Huang, K. Zhang, S. Maharjan, Y. Zhang, Blockchain empowered asynchronous federated learning for secure data sharing in internet of vehicles, *IEEE Trans. Veh. Technol.* 69 (4) (2020) 4298–4311.
- [22] J. Weng, J. Weng, J. Zhang, M. Li, Y. Zhang, W. Luo, Deepchain: auditable and privacy-preserving deep learning with blockchain-based incentive, *IEEE Trans. Dependable Secure Computing* 18 (5) (2019) 2438–2455.
- [23] J. Kang, Z. Xiong, D. Niyato, S. Xie, J. Zhang, Incentive mechanism for reliable federated learning: a joint optimization approach to combining reputation and contract theory, *IEEE Internet Things J.* 6 (6) (2019) 10700–10714.
- [24] X. Xu, L. Lyu, Towards Building a Robust and Fair Federated Learning System, *arXiv preprint arXiv:2011.10464*.
- [25] M. Shayan, C. Fung, C. J. Yoon, I. Beschastnikh, Biscotti: A Ledger for Private and Secure Peer-To-Peer Machine Learning, *arXiv preprint arXiv:1811.09904*.
- [26] P. Blanchard, E.M. El Mhamdi, R. Guerraoui, J. Stainer, Machine learning with adversaries: Byzantine tolerant gradient descent, in: *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 2017, pp. 118–128.
- [27] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine, D. Lewin, Consistent hashing and random trees: distributed caching protocols for relieving hot spots on the world wide web, in: *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing*, 1997, pp. 654–663.
- [28] P. Paillier, Public-key cryptosystems based on composite degree residuosity classes, in: *International Conference on the Theory and Applications of Cryptographic Techniques*, Springer, 1999, pp. 223–238.
- [29] B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: *Artificial Intelligence and Statistics*, PMLR, 2017, pp. 1273–1282.
- [30] B. Zhao, K. Fan, K. Yang, Z. Wang, H. Li, Y. Yang, Anonymous and privacy-preserving federated learning with industrial big data, *IEEE Transactions on Industrial Informatics* 17 (9) (2021) 6314–6323.
- [31] M. Mathias, R. Timofte, R. Benenson, L. Van Gool, Traffic sign recognition how far are we from the solution?, in: *The 2013 International Joint Conference on Neural Networks (IJCNN)* IEEE, 2013, pp. 1–8.
- [32] S. Caldas, S. M. K. Duddu, P. Wu, T. Li, J. Konečný, H. B. McMahan, V. Smith, A. Talwalkar, Leaf: A Benchmark for Federated Settings, *arXiv preprint arXiv: 1812.01097*.